AI-Enabled Developer

Module A – Model Fundamentals

Level 1: Foundations · Topics B01–B08

The mental models an engineer needs to debug a bad output instead of re-rolling the dice: how tokens are counted and charged, what the context window really is, why it degrades, and how to price a feature before you build it.

8	170	8	10
topics	minutes	concept diagrams	role profiles

© 2026 All rights reserved by Smart Working Solutions Ltd.

LEVEL 1: FOUNDATIONS

Module A at a glance

Eight topics, eight concept diagrams, roughly a half-day. Every topic slide states what the learner can do afterwards.

B01	**Who the course is for** Diagram · side panel	10 min
B02	**Human in the loop** Diagram · side panel	15 min
B03	**Tokens and tokenization cost** Diagram · full width	25 min
B04	**Context window and statelessness** Diagram · side panel	20 min
B05	**Context rot** Diagram · full width	30 min
B06	**Reasoning vs fast modes** Diagram · side panel	25 min
B07	**Multimodal input** Diagram · side panel	20 min
B08	**Napkin cost math** Diagram · full width	25 min

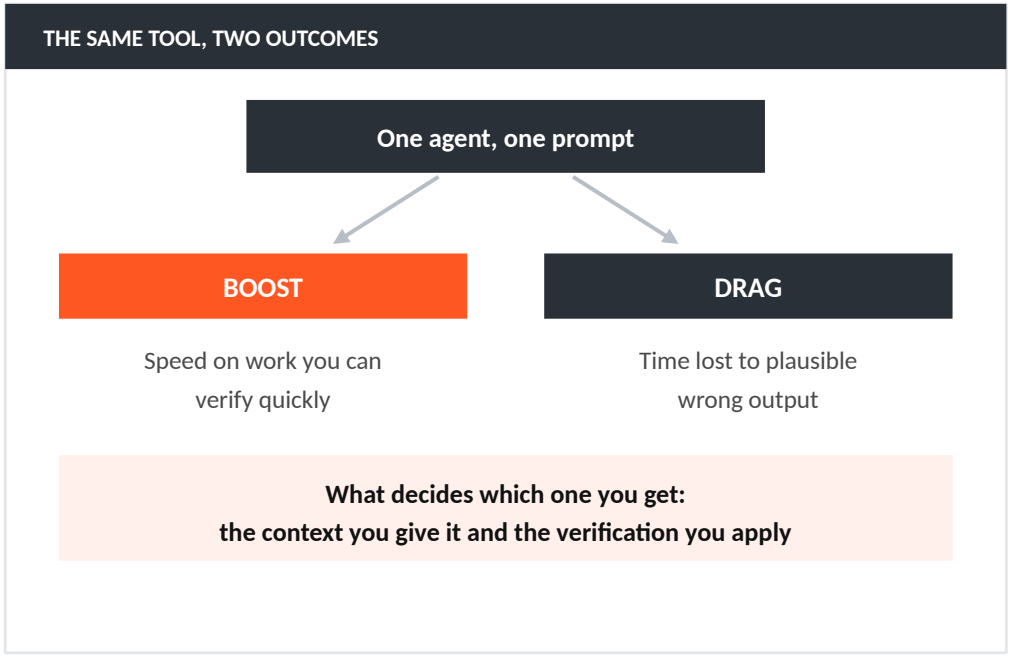
Role chips on every topic slide: FS Full-stack · BE Backend · FE Frontend · QA · DO DevOps/SRE · DE Data Eng · DS Data Sci/ML · PB Product/BA · SEC Security · EM Tech Lead

© 2026 All rights reserved by Smart Working Solutions Ltd.

Who the course is for

OUTCOME Recognise the difference between AI boost and AI drag — and what decides which one you get.

- Not a beginner course. It assumes you read code, run tests and review diffs.
- **Boost** is speed on work you can verify quickly — scaffolding, mechanical refactors, unfamiliar syntax.
- **Drag** is time lost to plausible wrong output you did not catch, and the bug that surfaces six weeks later.
- The same tool produces both. The variable is not the model — it is context and verification.



EXERCISE Name one recent task where AI clearly saved you time, and one where it clearly cost you time.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Human in the loop

OUTCOME Explain why fluency tells you nothing about correctness — and where responsibility sits.

- It is not lying. Lying requires knowing the truth. It generates the most plausible continuation.
- "A payments service has a method called `charge_customer`" is extremely plausible — which is why it gets written.
- **Fluency and correctness come from the same machinery**, so fluency is no signal at all.
- Ownership does not transfer. "The AI wrote it" is not a defence in a post-mortem, a review or an audit.
- The standard for the week: **if you cannot explain a line, you cannot merge it.**

WHAT IT IS ACTUALLY DOING

Predicting the next most plausible token — not looking anything up

```
payment = self.gateway.
```

```
charge_customer(        ← does not exist
process_payment(
createCharge(
submit(
```

Plausible is not the same as real

EXERCISE Run a real ticket through a model with no context. Identify what it invented, and ask whether you would have caught it in a 400-line diff.

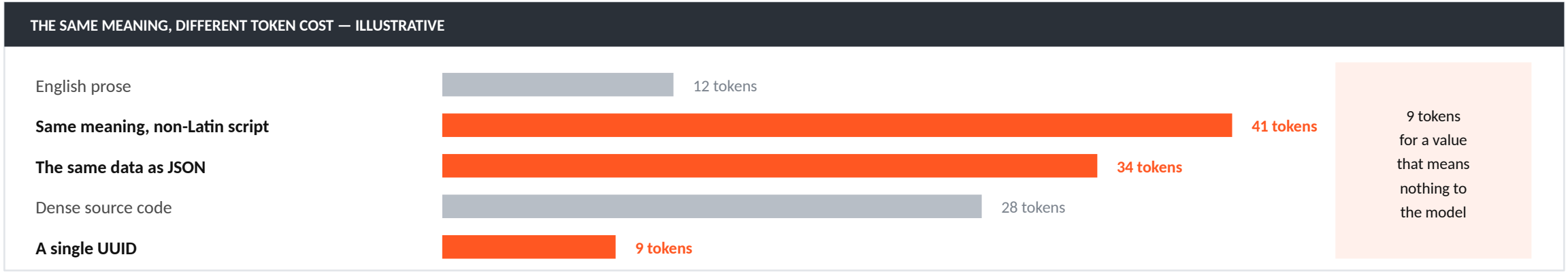
ROLES: FS · BE · FE · QA · DO · DE · DS · PB · SEC · EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Tokens and tokenization cost

OUTCOME Estimate token cost for a workload and explain why it differs by language and format.

- Roughly three quarters of an English word per token — true for English prose and misleading everywhere else.
- **Non-Latin scripts cost several times more for identical meaning.** Price a multilingual product off English testing and you are wrong by a factor, not a percentage.
- JSON and dense code tokenize badly: every brace, quote and colon spends money. A UUID costs nine tokens and means nothing.
- The cheapest place to compress is almost never the instructions — it is the data pasted underneath them.



EXERCISE Tokenizer bake-off: the same meaning across English, a non-Latin script, JSON, dense code and a UUID. Compute cost per 1,000 requests.

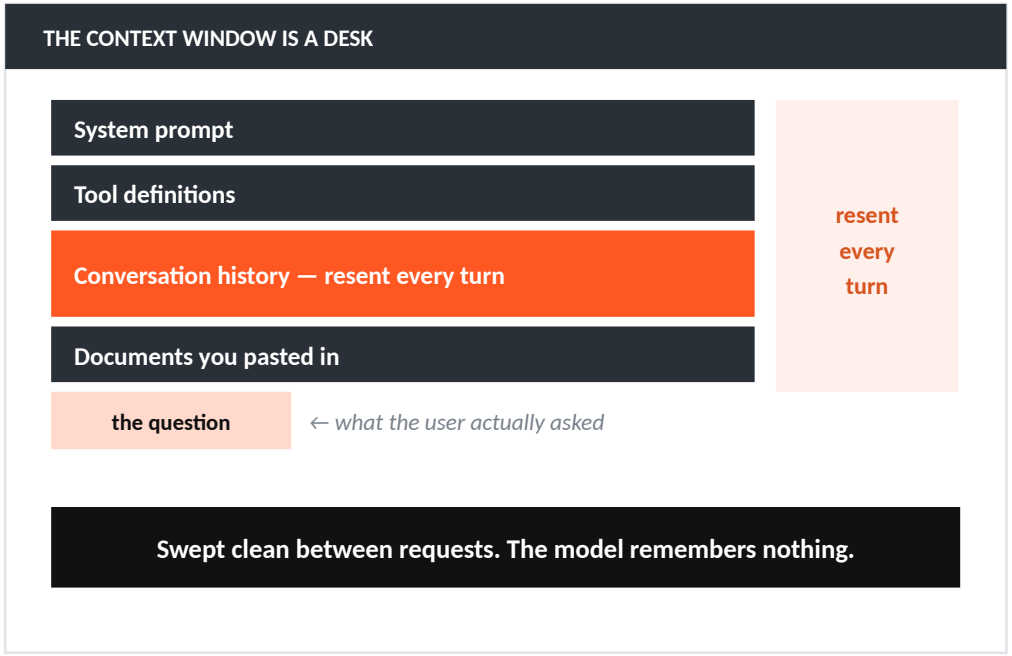
ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Context window and statelessness

OUTCOME Explain why conversation history is resent every turn, and where state must live.

- **The model has no memory.** A conversation is an illusion you construct by resending the entire history every turn.
- Cost grows with conversation length — turn ten pays for turns one through nine.
- **State management is your job**, not the model's: where history lives, what you truncate, what you summarise and when.
- Every "why is our AI bill so high" investigation ends on this slide.



EXERCISE Trace a 10-turn conversation and calculate the cumulative token cost of turn 10.

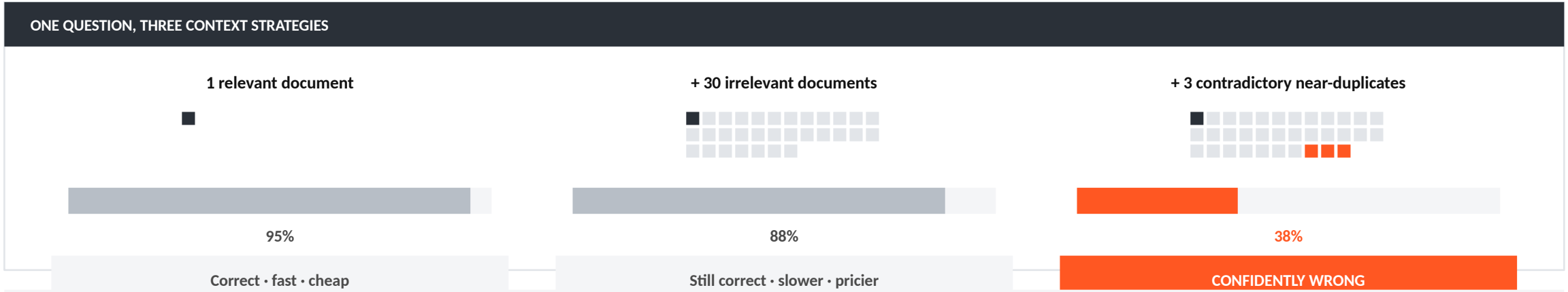
ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Context rot

OUTCOME Diagnose degraded output caused by an overfilled or contradictory context.

- Performance degrades as you fill the window — gradually, unpredictably, and **worst on the detail buried three screens down.**
- **Contradictory near-duplicates are the killer.** The model picks one silently and sounds completely confident about it.
- Missing information often produces a hedge you notice. Contradictory information produces a confident wrong answer you do not.
- Bigger windows did not end context engineering — they made it possible to fail more expensively. **More context is not better context.**



EXERCISE Run one question three ways: relevant document alone; plus 30 irrelevant; buried with 3 contradictory near-duplicates. Record accuracy and latency.

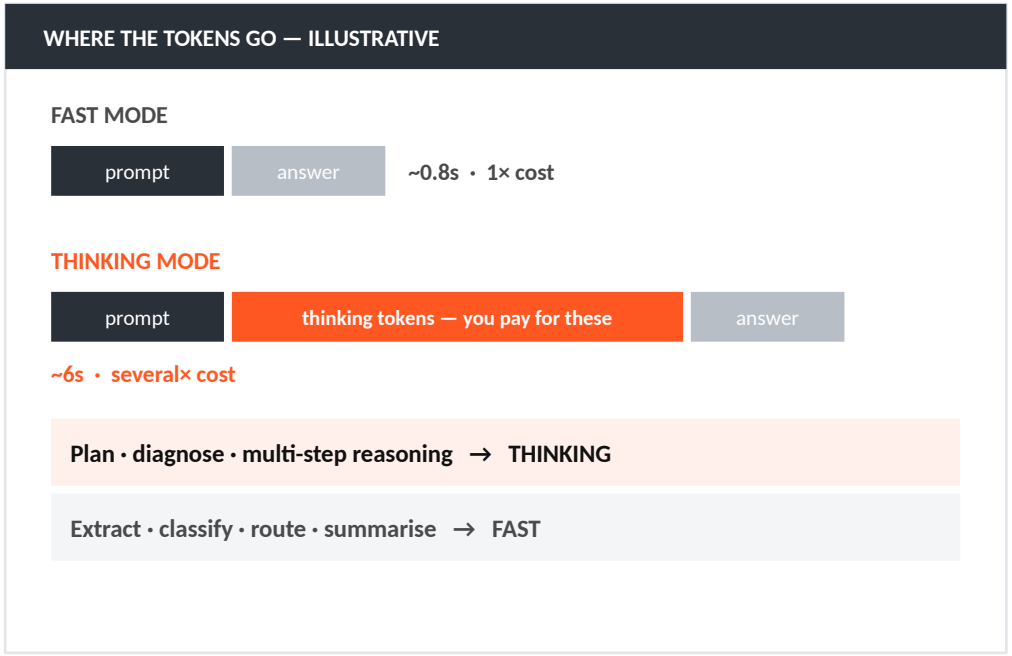
ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Reasoning vs fast modes

OUTCOME Choose thinking or fast mode per task with a cost and latency justification.

- Thinking modes spend extra tokens reasoning before answering — and you pay for those tokens.
- They genuinely help on planning, multi-step reasoning and hard debugging.
- On simple classification they are pure waste. You are paying a philosopher to read a barcode.
- **Thinking to plan and diagnose. Fast to extract and classify.**
- The moment you build a multi-step pipeline this stops being a preference and becomes an architecture decision.



EXERCISE Run one planning task and one classification task in both modes. Compare quality, latency and cost, then write the rule of thumb.

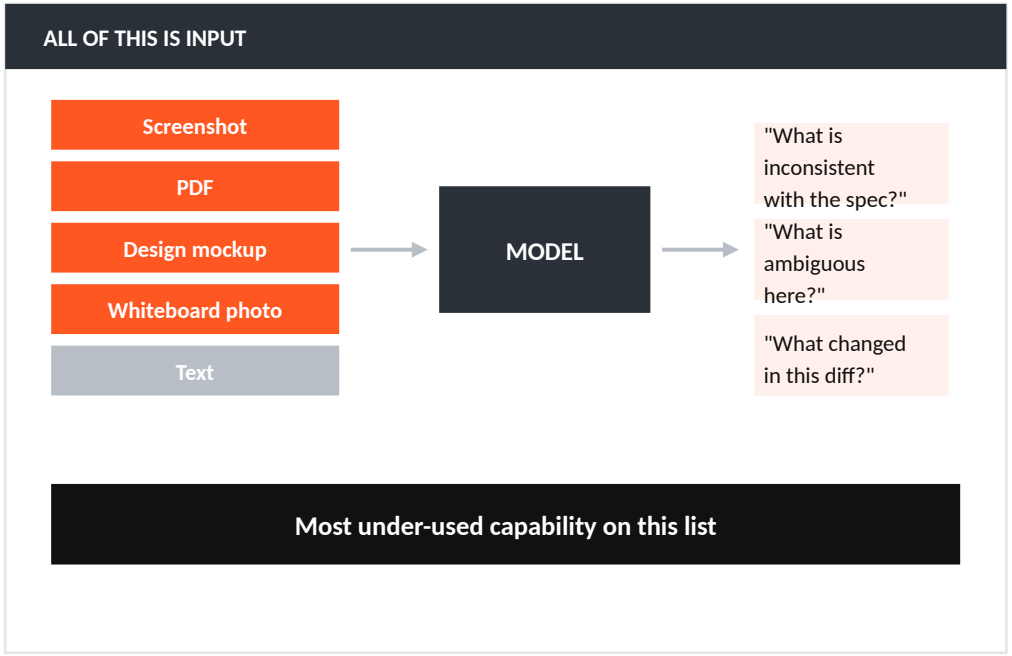
ROLES: FS, BE, QA, DO, DE, DS, EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Multimodal input

OUTCOME Use screenshots, PDFs and mockups as first-class model input.

- Images are first-class input, not a novelty. Screenshots, PDFs, design exports, photos of a whiteboard.
- For QA this is the sleeper capability: paste a broken layout next to the spec and ask what is inconsistent.
- For product and BA: paste the design export next to the acceptance criteria and ask what is ambiguous.
- Visual diff triage becomes an explanation you can read rather than four hundred red pixels.



EXERCISE Paste a screenshot of a broken UI next to the spec and ask what is inconsistent. QA variant: triage a visual diff.

ROLES: **FS** | BE | **FE** | **QA** | DO | DE | **DS** | **PB** | SEC | EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Napkin cost math

OUTCOME Price a feature before building it using volume × tokens × rate.

- Volume × tokens-in × rate, plus volume × tokens-out × rate. That is the whole model.
- **Output tokens dominate the bill despite being a fraction of the volume** — and in agentic workloads it is far more lopsided.
- So you optimise output length and step count first. Not your system prompt.
- Product and BA people who can do this arithmetic become the most credible person in a prioritisation meeting.

ONE FIFTH OF THE VOLUME. HALF THE BILL.

`50,000 tickets / month · ~2,000 tokens in · ~400 tokens out`



EXERCISE Whiteboard the monthly cost of a 50,000-ticket/month assistant. Identify which lever — input, output or steps — dominates the bill.

ROLES: **FS** **BE** FE QA **DO** DE DS **PB** SEC **EM**

© 2026 All rights reserved by Smart Working Solutions Ltd.

PLANNING REFERENCE

Module A exercises

Exercises now live here rather than on each topic slide. Each is repeated in the speaker notes of its topic.

ID	Topic	Exercise	Min
B01	**Who the course is for**	Name one recent task where AI clearly saved you time, and one where it clearly cost you time.	10
B02	**Human in the loop**	Run a real ticket through a model with no context. Identify what it invented, and ask whether you would have caught it in a 400-line diff.	15
B03	**Tokens and tokenization cost**	Tokenizer bake-off: the same meaning across English, a non-Latin script, JSON, dense code and a UUID. Compute cost per 1,000 requests.	25
B04	**Context window and statelessness**	Trace a 10-turn conversation and calculate the cumulative token cost of turn 10.	20
B05	**Context rot**	Run one question three ways: relevant document alone; plus 30 irrelevant; buried with 3 contradictory near-duplicates. Record accuracy and latency.	30
B06	**Reasoning vs fast modes**	Run one planning task and one classification task in both modes. Compare quality, latency and cost, then write the rule of thumb.	25
B07	**Multimodal input**	Paste a screenshot of a broken UI next to the spec and ask what is inconsistent. QA variant: triage a visual diff.	20
B08	**Napkin cost math**	Whiteboard the monthly cost of a 50,000-ticket/month assistant. Identify which lever — input, output or steps — dominates the bill.	25

© 2026 All rights reserved by Smart Working Solutions Ltd.

END OF MODULE A

What they can do now — and what comes next

AFTER MODULE A

- Estimate the token cost of a workload, and explain why it differs by language and format.
- Explain statelessness, and say where conversation state must actually live.
- Diagnose a bad output as context rot rather than re-rolling the dice.
- Choose a reasoning mode per task with a cost and latency justification.
- Price a feature on a napkin before anyone commits to building it.

NEXT: MODULE B

Context engineering — prompt structure, output contracts, few-shot at the edge, the four operations (write, select, compress, isolate), and the anti-patterns to stop using today.

Module A explained why context matters. Module B is how you control it.

You own every line you merge — whether you typed it or not.

© 2026 All rights reserved by Smart Working Solutions Ltd.